UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Chapter 3 : Principles Software Architecture

Innovating Solutions

UTMSPACE ESTD 1993 (931427-K)

SOFTWARE ARCHITECTURE

The foundational software design activity that evaluates and translates software requirements (both functional and non-functional) into a collection of design elements that specify structural and behavioral aspects of the major components of the system.



Foundational Design

Software architecture provides the groundwork essential for meeting requirements.

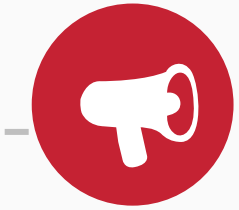
This applies to both functional and non-functional requirements



Requirement & Design Element

Requirements is a tricky business! Inexperienced engineers tend not to see the many traps behind the requirements effort.

Avoid ambiguous requirements.



Design Quality & Construction

Architectural work focuses on the major components, the quality properties, and services.

Result in a design that supports efficient detailed design and construction.

KEY TASK IN ARCHITECTURAL DESIGN

01

STAKEHOLDERS CONCERN

Stakeholders are persons, groups, or organizations that have a direct or indirect stake in the system.

02

APPROPRIATE ARCHITECTURAL

Different representations are required to evaluate certain properties of the system.

03

INFLUENCES OF ARCHITECTURAL

Impact of Architectural Decisions in Organization.
Impact of Architectural Decisions in Quality Attributes.

04

MAJOR COMPONENTS AND INTERFACES

Interfaces are defined for components residing within single physical nodes within a single or different process space, or for components residing on different physical nodes.

05

EVALUATING, VALIDATING THE ARCHITECTURE

Long and iterative process. Failure to do so can have significant impact in effort and cost incurred.

07

DESIGN SYNCHRONICITY

Design synchronicity is a measurement of the degree of how well the software implementation reflects its design, both in architectural form and detailed design form.

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

QUALITY ATTRIBUTES, STAKEHOLDER CONCERNED

Quality	Description
Usability	The degree of complexity involved when learning or using the system.
Modifiability	The degree of complexity involved when changing the system to fit current or future needs.
Security	The system's ability to protect and defend its information or information system.
Performance	The system's capacity to accomplish useful work under time and resource constraints.
Reliability	The system's failure rate.
Portability	The degree of complexity involved when adapting the system to other software or hardware environments.
Testability	The degree of complexity involved when verifying and validating the system's required functions.
Availability	The system's uptime.
Interoperability	The system's ability to collaborate with other software or hardware systems.

- Tactics for *Security*
  - ✓ Resisting Attacks
    - Authenticating users, Limit exposure, Limit access, only on need-to-know basis , etc.
  - ✓ Detecting Attacks
    - Intrusion detection
- Tactics for *Testability*
  - ✓ Event logging
    - Log data and operations throughout the system. Allow testers to enable/disable this feature so that when enabled, events are displayed in the console to give insight into the system's operations and data.
- Tactics for *Modifiability*
  - ✓ Localize changes
    - Modularization, abstraction, encapsulation
  - ✓ Prevention of ripple effects
    - Encapsulation, reduce coupling
- Tactics for *Availability*
  - ✓ Redundancy, Task monitor, Watchdog timer , etc.
- Tactics for *Performance*
  - ✓ Increase computation efficiency, reduce computational overhead, introduce concurrency, etc.

PROBLEM SOLVING DURING ARCHITECURE

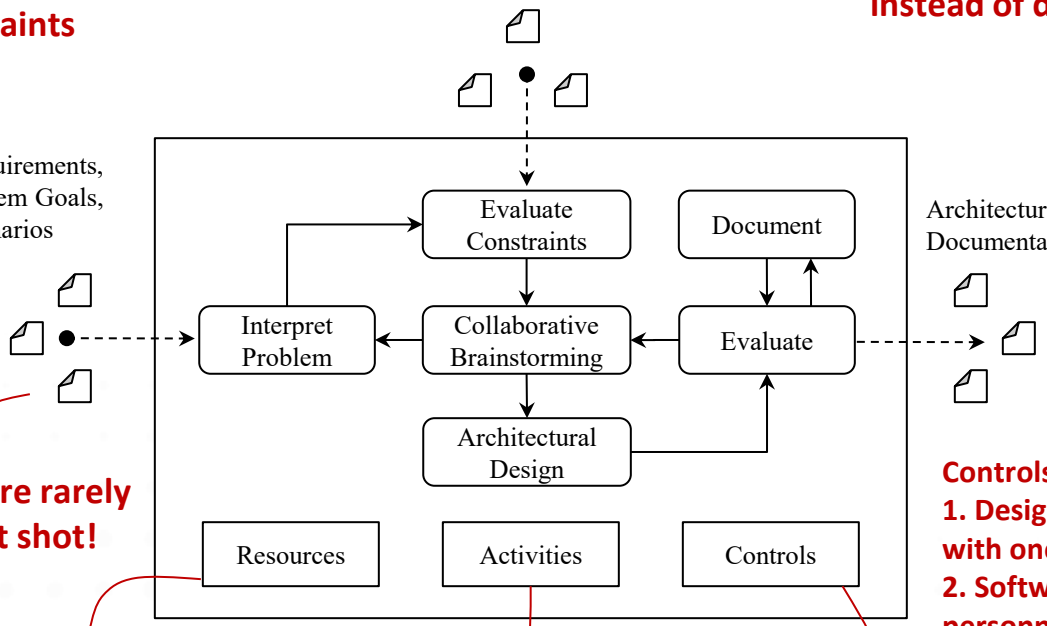
Other project related constraints

For example, we will reuse our home-grown class framework to leverage our expertise and develop faster!

Another example, we will buy and integrate xyz library into our product instead of developing it ourselves

Schedule Constraints,
Cost Constraints,
Quality Constraints

These define the problem and product constraints



Unfortunately, these are rarely complete on the first shot!

Development team, modeling software, IDEs, hardware, etc.

Tasks performed to process the requirements Example: Design Review

These dictate how, when, and where activities are performed!

Controls Examples:
1. Design reviews must be scheduled with one week of anticipation
2. Software Quality Assurance personnel must be present during review.
3. Review of material containing classified information must occur inside a vault

SOFTWARE ARCHITECTURE PROCESS

01. Understand and Evaluate Requirement

Elicit technique using scenarios.

Requirement classification, prioritization and negotiation.

Specific and Valid.

02. Design the Architecture

Architecture, design and tech stacks.

03. Evaluate the Architecture

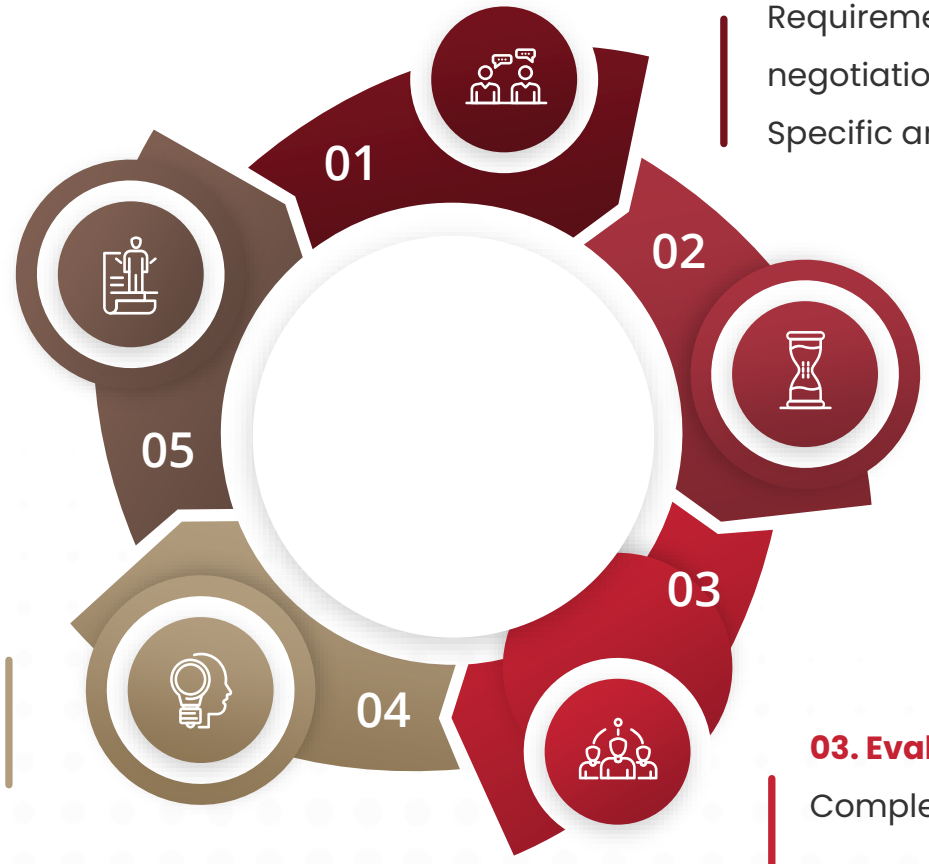
Complete and matched.

04. Document the Architecture

Upon completion all documented and signed of.

05. Monitor and Control Implementation

Development and monitoring.



Examples

Notice how the original requirement is broken into two complete requirements!

This is a good requirement in the sense that it is specific and correct.

Complete	Requirement
No	The software shall generate product reports.
Yes	The software shall generate product reports consisting of product description, picture, and price.
	Product reports shall be in PDF format.

However, if we know the requirements for product reports, then a complete version of this requirement specifies this information. This help disambiguate the notion of a product report so that it can be implemented easily in code.

Can you meet this requirement today?

Attainable	Requirement
No	The software shall execute on all future operating systems.
Yes	The software shall execute on the Microsoft Windows 7 platform.

Verifiable	Requirement
No	The system shall maximize communication speed.
Yes	The system's data rate shall be no less than 1Mbps.

Examples

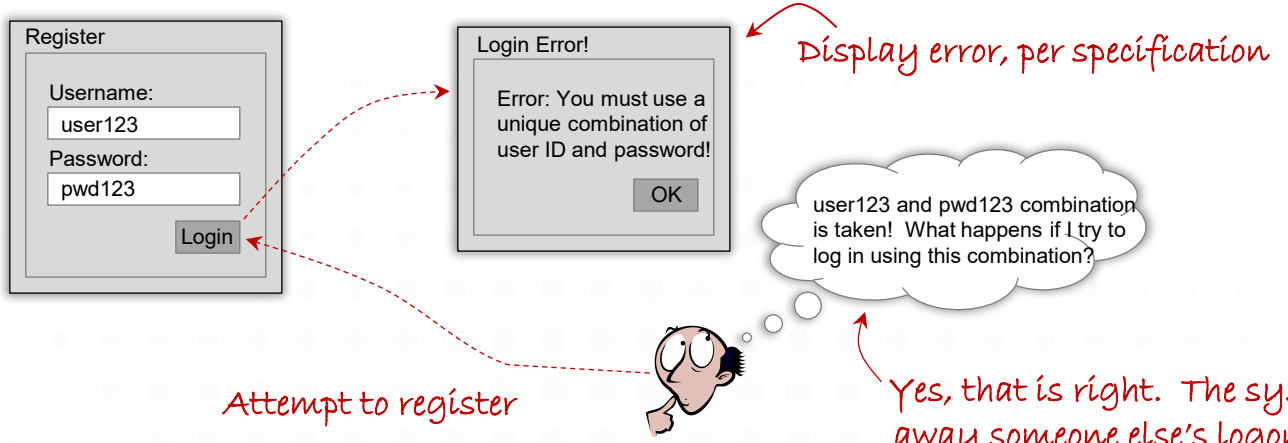
Search the database for what?

What does this mean?

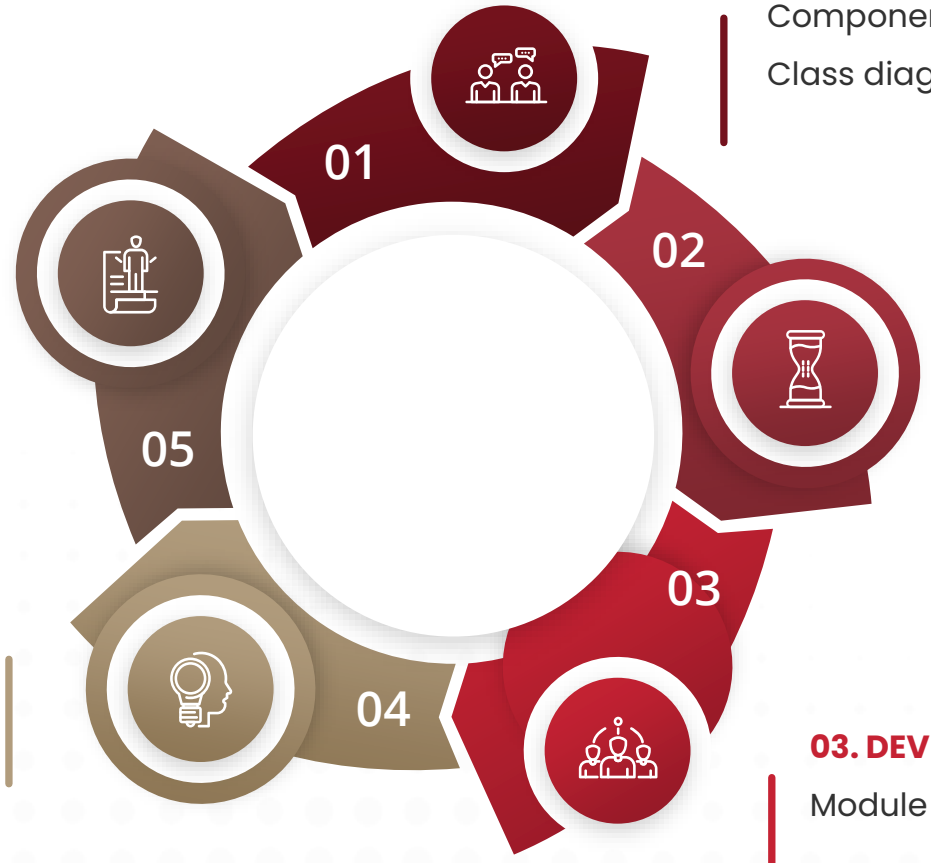
Specific	Requirement
No	The software shall search the database.
Yes	The software shall search for a product using the product ID.
No	The software shall be secure.
Yes	The software shall authenticate users with user ID and password.
No	The software shall be secure and fast.
Yes	The software shall authenticate users with user ID and password.
	Server acknowledgment message shall be sent within 1/2 second from the time a request is received.

Secure and fast?

These two requirements are specific and provide the information required to determine what secure and fast mean!



DESIGNING WITH THE 4 + 1 VIEW



01. LOGICAL VIEW

Component diagrams

Class diagrams

02. PROCESS VIEW

Sequence diagrams.

Communication diagrams.

03. DEVELOPMENT VIEW

Module or package diagrams.

04. PHYSICAL VIEW

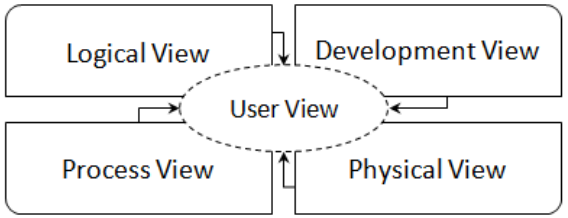
Deployment diagrams.

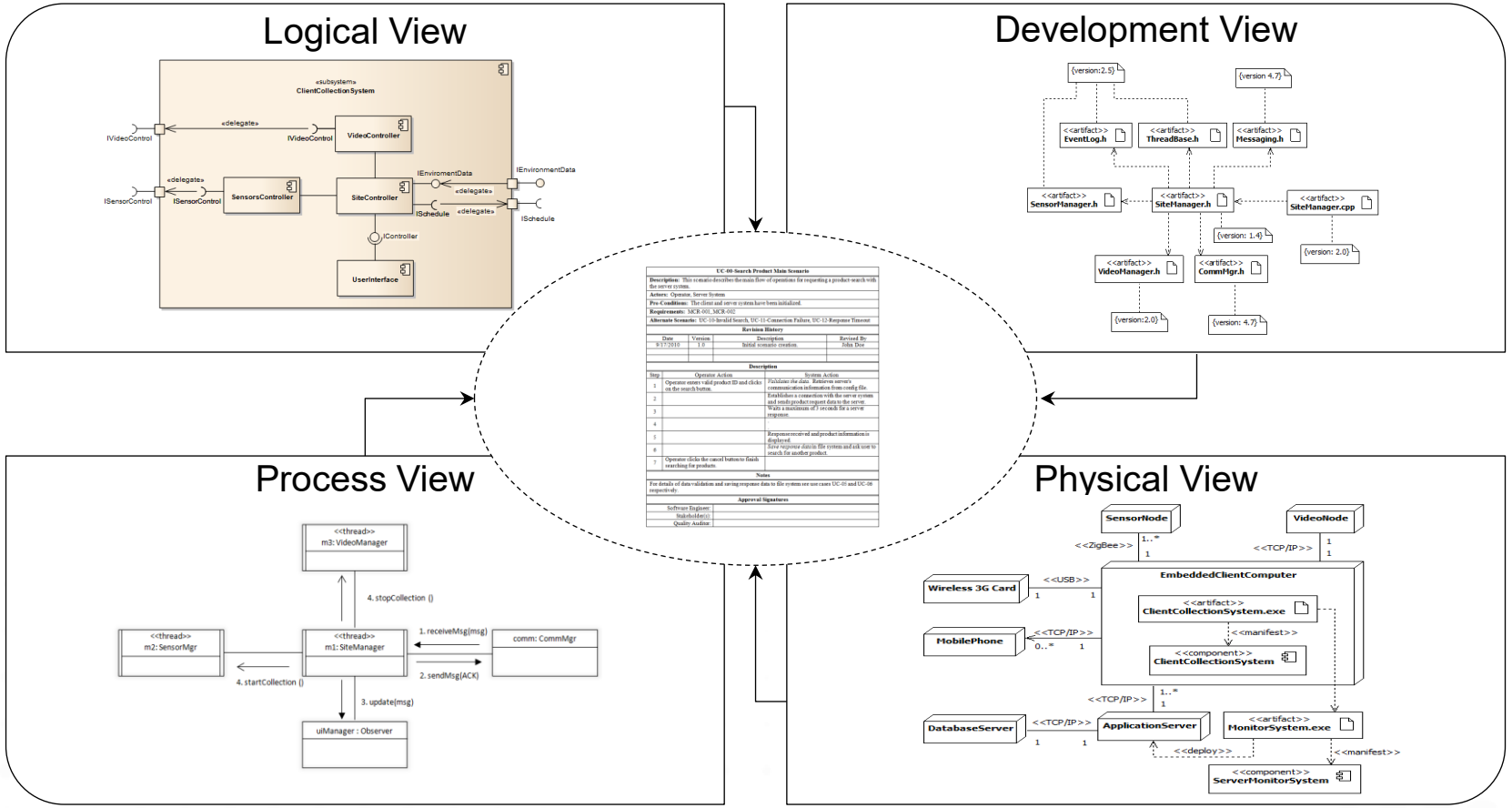
Infrastructure diagrams.

05. USER VIEW

Use cases and scenarios.

THE 4 + 1 VIEW


Logical View
Development View
User View
Process View
Physical View


Logical View
«subsystem»
ClientCollectionSystem
IVideoControl
«delegate»
VideoController
SensorsController
SiteController
IEnvironmentData
ISchedule
IController
UserInterface
Development View
{version:2.5}
{version 4.7}
<<artifact>>
EventLog.h
ThreadBase.h
Messaging.h
SensorManager.h
SiteManager.h
SiteManager.cpp
{version: 1.4}
{version: 2.0}
VideoManager.h
CommMgr.h
Process View
<<thread>>
m3: VideoManager
4. stopCollection ()
m2: SensorMgr
m1: SiteManager
1. receiveMsg(msg)
comm: CommMgr
2. sendMsg(ACK)
4. startCollection ()
3. update(msg)
uiManager : Observer
Physical View
SensorNode
VideoNode
<<ZigBee>>
<<TCP/IP>>
Wireless 3G Card
<<USB>>
EmbeddedClientComputer
ClientCollectionSystem.exe
MobilePhone
<<manifest>>
ClientCollectionSystem
DatabaseServer
ApplicationServer
MonitorSystem.exe
<<deploy>>
ServerMonitorSystem



Thank you for your participation. I am very grateful to have the opportunity into this sharing session. If there any enquiries, feels free to contact me.

+013 509 9215

khatibsyarbini@utm.my

THANK YOU

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

In the Name of God for Mankind

utm.my univteknologimalaysia utmofficial